

COMP6252 Coursework 1

Music Genre Classification with Six Neural Network Architectures

University of Southampton
Student: Isha Girish Kulkarni
Student ID: 37642456
igk1g25@soton.ac.uk

Abstract

This report compares six neural network models on the GTZAN music genre dataset. Net1 to Net4 use MEL spectrogram images and cover a fully connected baseline along with three CNN variants. Net5 and Net6 use MFCC sequences passed through a Bidirectional LSTM, with Net6 also trained on additional samples from a GAN. The best test accuracy of 66.34% comes from Net3, which adds Batch Normalisation to the CNN. Adding BatchNorm gave the biggest improvement. Adam with cosine annealing trained faster than RMSprop, and the unconditional GAN augmentation in Net6 actually reduced accuracy slightly because the synthetic samples were not tied to any class label.

1 Introduction

Music genre classification is a common task in audio machine learning but it is not an easy one. Many genres share similar musical features. Rock and metal both rely on distorted guitars, jazz and blues both use swing rhythms, and pop, disco and hip hop sit in similar tempo ranges. A good model needs to look at detail in the spectrogram or audio rather than just simple statistics like loudness or beat speed.

The GTZAN dataset has 1,000 audio clips of 30 or more seconds each, split across 10 genres (blues, classical, country, disco, hip hop, jazz, metal, pop, reggae and rock). The dataset comes with both raw .wav files and MEL spectrogram PNG images. One audio file jazz.00054.wav is corrupted, so 999 samples are usable. Because the dataset is small, regularisation and a careful train/test split are important. This work uses both modalities. Net1 to Net4 work on the spectrogram images and let me compare a fully connected network against CNNs, look at the effect of Batch Normalisation, and compare two optimisers (Adam and RMSprop). Net5 and Net6 work on MFCC sequences with a Bidirectional LSTM. Net6 adds GAN generated MFCC samples to the training set to test whether synthetic data can help when real data is limited.

2 Data Preprocessing

Image pipeline (Net1 to Net4). The spectrogram PNGs are loaded using torchvision.ImageFolder and resized to 180×180 pixels. Pixels are normalised channel wise using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma =$

$[0.229, 0.224, 0.225]$). Spectrograms are not really natural images, but this normalisation puts the values in a reasonable range for ReLU activations. The 999 images are split 70/20/10 into 699 training, 199 validation and 101 test samples using random_split with seed 42.

Audio pipeline (Net5 and Net6). Raw .wav files are loaded with the soundfile library, and the corrupted file is skipped using sf.info(). Each audio clip is split into $N_{\text{seg}} = 10$ equal windows. For each window, 40 MFCC coefficients are computed (sample rate 22,050 Hz, FFT size 2048, hop length 512, 128 mel bins) and then averaged over time. This gives a (10, 40) tensor per clip. Each sample is standardised to zero mean and unit variance, and the same 70/20/10 split is used.

3 Network Architectures and Training

3.1 Net1: Fully Connected Network

Each $3 \times 180 \times 180$ image is flattened to a vector of size 97,200 and passed through two hidden layers: $97,200 \rightarrow 512 \rightarrow 256 \rightarrow 10$. The first layer alone has roughly 50.2M parameters. Each hidden layer uses BatchNorm1d, ReLU and Dropout(0.4) to reduce overfitting. Training uses Adam with learning rate 10^{-3} and weight decay 10^{-4} , plus a StepLR scheduler ($\gamma = 0.5$, step 20 for the 50 epoch run and step 30 for the 100 epoch run).

3.2 Net2: Convolutional Network

Net2 uses two convolutional blocks with 3×3 kernels and padding 1. Block 1 has Conv($3 \rightarrow 32$), ReLU, Conv($32 \rightarrow 64$), ReLU, MaxPool(2×2) and Dropout2d(0.25). Block 2 has Conv($64 \rightarrow 128$), ReLU, Conv($128 \rightarrow 256$), ReLU, MaxPool and Dropout2d(0.25). An AdaptiveAvgPool reduces the feature map to 1×1 and the classifier is FC($256 \rightarrow 128$), ReLU, FC($128 \rightarrow 10$). Training uses Adam (lr 10^{-3} , weight decay 10^{-4}) with StepLR ($\gamma = 0.5$).

3.3 Net3: CNN with Batch Normalisation

Net3 is the same as Net2 but with BatchNorm2d added after every Conv2d layer and before each ReLU. Normalising the feature maps within each mini batch reduces internal covariate shift, makes training more stable and gives some implicit regularisation. The optimiser is Adam (lr 5×10^{-4} , weight

decay 10^{-4}) with CosineAnnealingLR, which slowly decays the learning rate towards zero so the model settles near the minimum.

3.4 Net4: Net3 with RMSprop

Net4 uses the same architecture as Net3 but swaps Adam for RMSprop (lr 10^{-3} , weight decay 10^{-4} , default $\alpha = 0.99$). RMSprop keeps a running average of squared gradients and divides each update by the square root of that average, giving per parameter learning rate adaptation. Unlike Adam, it does not use a momentum term. The scheduler is StepLR ($\gamma = 0.5$).

3.5 Net5: Bidirectional LSTM

The (10, 40) MFCC tensor is fed into a single layer Bidirectional LSTM with hidden size 64. Total parameter count is 63,178. Reading the sequence forwards and backwards lets the model use both onset and decay information. The 10 output vectors of size 128 are mean pooled into one 128 dim summary, which then goes through FC(128 $\rightarrow$ 64), ReLU, Dropout(0.4), FC(64 $\rightarrow$ 10). Training uses Adam (lr 10^{-3} , weight decay 10^{-4}) and StepLR (step 30, $\gamma = 0.5$). Early stopping monitors validation accuracy with patience 20 and a maximum of 150 epochs. Training stopped at epoch 75.

3.6 Net6: BiLSTM with GAN Augmentation

A GAN is trained directly in MFCC feature space. The generator maps a 64 dim Gaussian noise vector to a 400 dim flat MFCC sample through layers 64 $\rightarrow$ 128 $\rightarrow$ 256 $\rightarrow$ 400, using LeakyReLU and BatchNorm1d, with a Tanh output to keep values in $[-1, 1]$. The discriminator goes 400 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 1 with Dropout(0.3) and a Sigmoid. To keep training stable I used label smoothing (real labels 0.9, fake labels 0.1), a 2:1 generator to discriminator update ratio, and asymmetric learning rates (lr_D = 10^{-4} , lr_G = 2×10^{-4} , Adam $\beta_1 = 0.5$). The GAN was trained for 80 epochs. Then 699 synthetic (10, 40) samples were generated, scaled back to the real data range, and added to the real training set to make 1,398 samples total. Net6 has the same architecture as Net5 and trained on this larger set with early stopping at epoch 90.

4 Results

Table 1 shows the test accuracy of all six models. Two results stood out. First, Net1 actually got worse with longer training, dropping from 47.52% to 36.63%. Second, Net3 reached its best at 50 epochs (66.34%) and slipped slightly at 100 epochs. Net5 (BiLSTM on audio) reached 64.36% with only 63K parameters, which is similar to Net3 at 100 epochs. Figures 1, 2 and 3 show the training and validation accuracy curves.

Model	50 ep	100 ep
Net1 Fully Connected	47.52%	36.63%
Net2 CNN	42.57%	50.50%
Net3 CNN + BatchNorm	66.34%	64.36%
Net4 CNN + BN + RMSprop	58.42%	64.36%
Net5 BiLSTM (audio)	64.36% (ep. 75)	
Net6 BiLSTM + GAN	61.39% (ep. 90)	

Table 1: Test accuracy on Neural Networks.

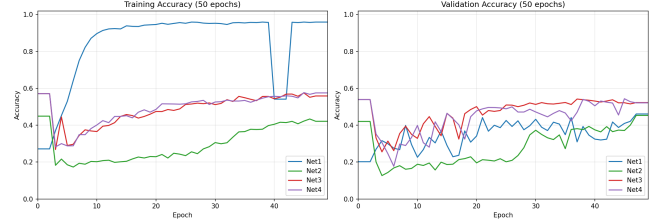


Figure 1: Train (left) and validation (right) accuracy for Net1 to Net4 at 50 epochs.

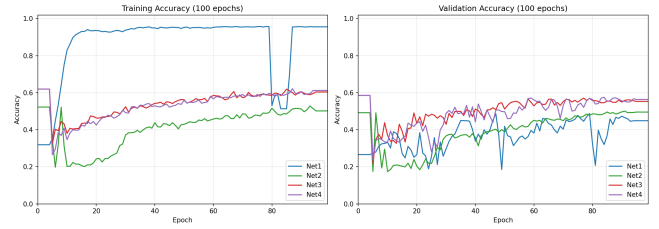


Figure 2: Train (left) and validation (right) accuracy for Net1 to Net4 at 100 epochs.

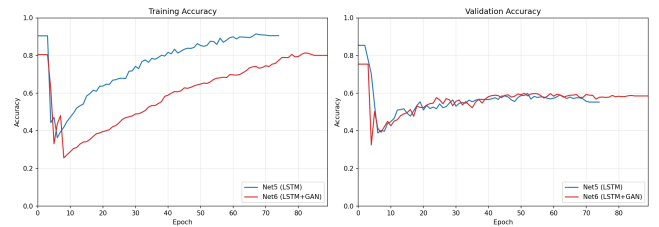


Figure 3: Train (left) and validation (right) accuracy for Net5 and Net6.

5 Discussion

FC vs. CNN (Net1 vs. Net2). Net1 overfits badly. Training accuracy reaches close to 99% while test accuracy drops from 47.52% at 50 epochs to 36.63% at 100 epochs, a fall of about 10.89%. The first FC layer alone has $97,200 \times 512 \approx 50M$ parameters, but there are only 699 training samples. Dropout and BatchNorm slow the overfitting but cannot stop it. The model also has no spatial inductive bias, so it treats each

pixel as independent and ignores the local frequency and time structure of a spectrogram. Net2 avoids both problems. Convolutional weight sharing brings the parameter count down a lot and gives translation invariance. Test accuracy goes up steadily from 42.57% to 50.50%.

Batch Normalisation (Net2 vs. Net3). BatchNorm gave the largest gain in the whole study, 66.34% against 42.57% at 50 epochs, an improvement of around 23.77%. Normalising feature maps within each mini batch makes the loss surface smoother and acts as a mild regulariser. The small drop to 64.36% at 100 epochs is most likely a bit of late stage overfitting once the cosine schedule pulls the learning rate close to zero.

Optimiser comparison (Net3 vs. Net4). At 100 epochs both reach 64.36%, but Net3 with Adam and cosine annealing already hits 66.34% by epoch 50, while Net4 with RMSprop only manages 58.42% at the same point, a gap of 7.92 %. Adam uses a momentum term that helps in the early epochs, and the cosine schedule keeps the model close to its best validation score. The fact that both end up at the same final accuracy suggests that the architecture, not the optimiser, sets the ceiling here.

GAN augmentation (Net5 vs. Net6). Against my expectation, Net6 (61.39%) did slightly worse than Net5, a drop of about 2.97%. The reason is that the GAN was unconditional. The generator only sees noise, with no genre label, so the synthetic samples do not really belong to any class. I assigned them pseudo labels in a cyclic order (0, 1, ..., 9, 0, ...), which means the LSTM is essentially learning from mislabelled data. The GAN itself trained fine (D loss around 1.40 and G loss around 0.67 at convergence), so the issue is not training stability but a mismatch between the synthetic samples and their labels. A conditional GAN would solve this directly.

Image vs. audio modality (Net3 vs. Net5). Net5 reaches 64.36%, which matches Net3 at 100 epochs but stays about 1.98% below Net3's best (66.34% at 50 epochs). The interesting part is the cost. The MFCC input is roughly 243 times smaller than the spectrogram input (400 values vs. 97,200 values) and Net5 has only 63K parameters compared to around 0.5M for Net3. Because each of the 10 MFCC windows is averaged in time during preprocessing, the LSTM does not see fine onset detail. What it does see is how the average spectral envelope changes from window to window across the 30 second clip. So the takeaway is not that audio beats images, but that a much smaller temporal model can get close to a CNN on this dataset, which is useful when memory or compute is limited.

Overall. Architectural choices like weight sharing and BatchNorm matter more than training length or optimiser choice on a small dataset like GTZAN. Net3 has the best result at 66.34% in just 50 epochs, and Net5 shows that a small temporal model can do almost as well as a much larger CNN.

Training curves. The curves in Figures 1 and 2 show patterns that the final accuracy alone hides. Net1 saturates near 100% training accuracy in less than 10 epochs while valida-

tion accuracy stays near 40%, which is a clear sign of memorisation. Net2's curves rise smoothly and stay close together, so it is still learning at epoch 100 and could benefit from more training. Net3 and Net4 show closer alignment between training and validation, which fits with BatchNorm acting as a regulariser. Net4's validation curve is noisier than Net3's, which makes sense because RMSprop has no momentum term to smooth out mini batch variance.

Parameter efficiency. Net5 has 63,178 parameters. Net1 has roughly 50.2M parameters in its first FC layer alone, a ratio of around 800 times. Net2, Net3 and Net4 have roughly 0.5M parameters in the convolutional path. Net3 still beats Net1 by 18.71% at 50 epochs and 27.73 % at 100 epochs. So on a small audio dataset the parameter count is not a useful proxy for capacity. What matters is whether the parameters carry the right inductive bias.

Table 2: Final-epoch losses

Net	Config	Ep.	Train	Val
Net1	FC (2 hidden)	50	0.0153	1.8373
Net1	FC (2 hidden)	100	0.0104	2.4250
Net2	CNN	50	1.5310	1.6072
Net2	CNN	100	1.3374	1.4447
Net3	CNN + BN	50	1.2572	1.3474
Net3	CNN + BN	100	1.0701	1.2197
Net4	CNN + BN + RMSProp	50	1.1990	1.3217
Net4	CNN + BN + RMSProp	100	1.0556	1.2822
Net5	BiLSTM (MFCC)	75*	0.3059	1.7230
Net6	BiLSTM + GAN aug.	90*	0.3741	2.7383

*early-stopped

6 Limitations and Future Work

Dataset size. GTZAN has only 999 usable samples. The validation set has 199 samples, which makes the validation curve quite noisy, and the test set of 101 samples means a confidence interval of around ± 5 points at the accuracy levels seen here. Larger datasets would give more stable estimates.

Spectrogram resolution. I used 180×180 inputs to keep training time reasonable on CPU. A higher input resolution with a deeper CNN, such as a small ResNet, would probably get closer to the published GTZAN state of the art around 80%.

GAN augmentation. The Net6 GAN is unconditional, which is the main reason it added label noise. The clearest fix is a class conditional GAN that also takes a one hot genre label as input on both the generator and the discriminator. WGAN with gradient penalty would also help training stability and give a loss correlated with sample quality.

Cross validation. I used a single 70/20/10 split with a fixed seed, which is reproducible but high variance. A 5 fold cross validation would give mean and standard deviation values that are easier to compare against other results.

Part 2: Reflection on Diffusion Models for Generative Deep Learning

Topic and Why it Matters

Among the topics covered in the first four lab sessions, the one I found most worth reflecting on is diffusion models, introduced in Lab 3. They are currently the standard choice for high fidelity generative modelling and sit underneath systems such as Stable Diffusion, Imagen, DALL·E 3 and Sora. Their importance is no longer limited to creative image or video synthesis: they are also used in protein structure refinement (for example RFdiffusion), in 3D molecule generation for drug discovery, in medical image reconstruction from sparse MRI or CT acquisitions, and in data augmentation for clinical datasets where real samples are scarce.

This last point connects directly to my own research interests. I work on a wearable cardiac biosensor where labelled patient data is hard to obtain, and diffusion models offer a way to generate plausible physiological signals such as ECG, PPG and accelerometer traces to support classifier training. They are also attractive technically: they train more stably than GANs, cover modes more reliably, produce sharper outputs than VAEs, and decode in parallel rather than one token at a time as autoregressive models do. The fact that the same training recipe transfers across image, audio, video and 3D with relatively few changes is what makes them feel like a genuine general purpose framework rather than a single trick.

Current Deep Learning Technologies for the Topic

The core idea is straightforward. A fixed forward Markov chain corrupts a clean sample x_0 with Gaussian noise on a schedule $\{\alpha_t\}$, and a network $\epsilon_\theta(x_t, t)$ is trained to predict the noise at each timestep with a simple mean squared error loss. This loss is equivalent, up to constants, to a variational bound on the data log likelihood, which is part of why training is so much more stable in practice than the alternatives.

Around this core the field has built a fairly rich set of methods. DDPM is the original formulation. DDIM gives faster, more deterministic sampling. Score based models cast the problem as a stochastic differential equation and connect it to classical score matching. Latent Diffusion Models work in the compressed latent space of a VAE, which makes high resolution synthesis practical on standard hardware. Classifier free guidance lets a single network handle both conditional and unconditional generation. ControlNet adds structural conditioning on edges, depth or pose. Consistency models and rectified flow distil multi step samplers into one or two steps for real time use. On the architectural side, the community is moving from U Nets to Diffusion Transformers (DiT), which underpin Sora style video models. The same ideas appear in audio through DiffWave and Grad TTS.

What I Can and Cannot Yet Implement?

I am reasonably confident with the parts of this stack closest to a standard supervised loop: the forward noising process, a basic U Net denoiser trained with the simplified DDPM objective, a linear or cosine noise schedule, sinusoidal timestep embeddings, and a DDIM sampler with around 50 to 100 steps. I have tried this on small datasets such as MNIST and CIFAR 10, and I am comfortable adapting the loop to 1D physiological signals.

The harder components are the ones I cannot yet implement at the same level. Training a Latent Diffusion Model end to end requires first training a good VAE and then handling two interacting loss landscapes. Classifier free guidance is conceptually simple but easy to get wrong, since it needs conditional dropout in training and a tuned guidance scale at inference. Cross attention conditioning on text embeddings from a pretrained encoder such as CLIP or T5 is something I have read about in detail but not implemented. The difficulties here are mostly practical: compute cost, multi component engineering, and numerical stability of the reverse SDE at low noise levels. Distillation work like consistency models and rectified flow is also moving quickly enough that it does not feel settled.

Positive, Negative Impact, and Future Vision

On the positive side, diffusion models have made high quality generative tooling much more widely available, contributed to scientific discovery in protein design and medical imaging, and given researchers a principled way to augment scarce datasets. Their training stability and theoretical grounding make them more reliable than earlier generative families, and the open release of models such as Stable Diffusion has let smaller groups outside large industry labs use and study them seriously.

The negative side is harder to dismiss. They are used to produce deepfakes and non consensual imagery, can accelerate copyright laundering at scale, and the energy and water footprint of training and inference at frontier scale is increasingly hard to justify. Questions around attribution and the rights of the artists whose work appears in training sets are now being tested in court.

Looking ahead, I expect three directions to dominate: faster samplers that get useful quality in one or two steps; unified multimodal backbones across image, audio, video and 3D; and closer integration with scientific simulation, where the learned prior is combined with physical constraints. On the governance side, watermarking, provenance frameworks such as C2PA, and clearer training data licensing will probably matter as much in practice as the architectures. The direction I find most useful personally is diffusion becoming a routine tool inside medical AI pipelines, so smaller clinical groups can train robust models without proprietary datasets.